UniAccounts - Student Account Management

Practical Work (TP) - Design Patterns in Java

Work 1: the core and the creational patterns

Student(s): _____ | Group: _____ | Date: _____

1. Project overview

UniAccounts manages student accounts for a university. This first stage builds the **core**: a `StudentAccount` class (8 fields, 3 mandatory and 5 optional), one central `AccountRepository` and one `AuditLogger`. Three creational patterns are used, each at the place where it solves a real problem: **Builder** to create the account, **Singleton** to guarantee a single repository and a single audit log, and **Prototype** to create the accounts of a whole batch by cloning a template.

Pattern	Classes	Package
Builder	StudentAccount.Builder (builds StudentAccount)	model
Singleton	AccountRepository, AuditLogger	repository, audit
Prototype	Prototype<T>, StudentAccount.copy() / copyFor()	prototype, model

2. Class diagram (UML)

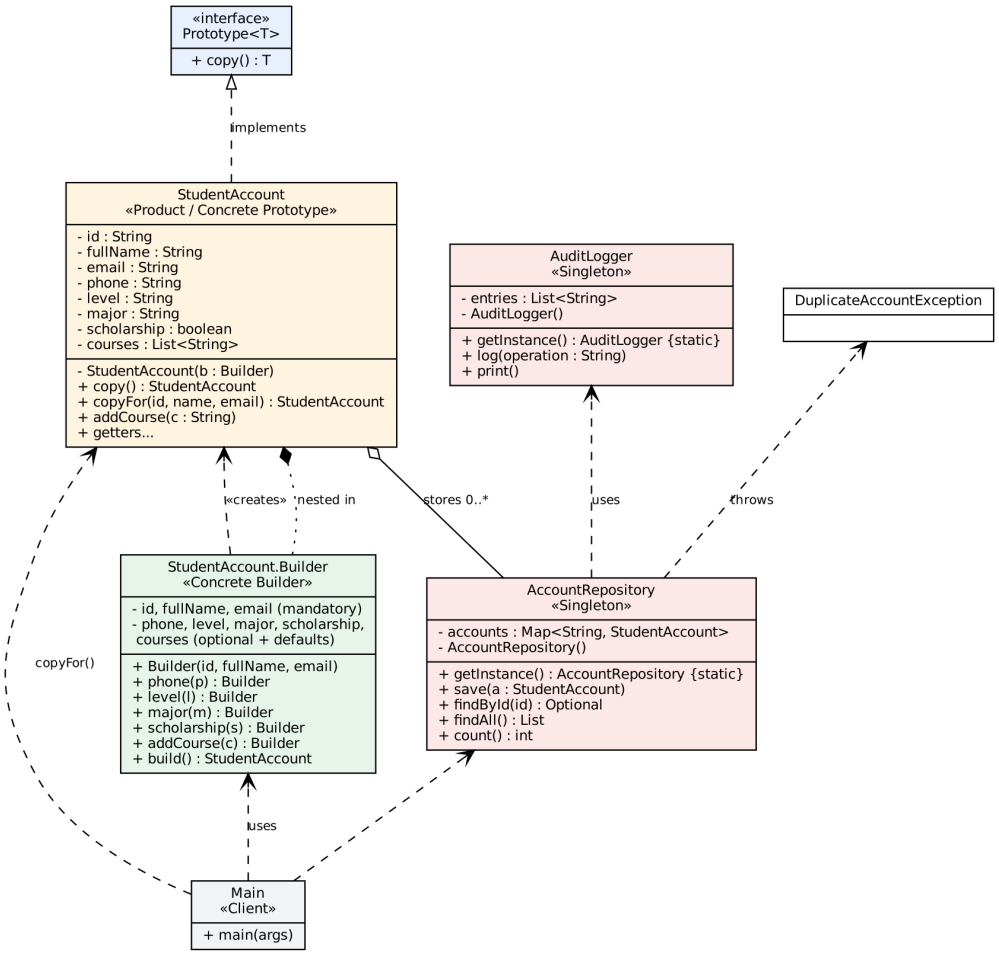


Figure 1 - UML class diagram: the three patterns and their relations. *StudentAccount* is the Product of the Builder and the Concrete Prototype; the two Singletons are used by the Client (*Main*); the repository uses the logger.

3. The patterns: where, why, and what if absent

3.1 Builder - StudentAccount.Builder

Where? The nested class `StudentAccount.Builder`. The mandatory fields (*id, fullName, email*) are passed to the Builder constructor; optional fields (*phone, level, major, scholarship, courses*) are set through chained methods, each with its default value (null, L1, false, empty list). `build()` checks all the rules and only then creates the account through a private constructor.

Problem solved. With eight fields, five of them optional, a single constructor with eight parameters is unreadable and error-prone: two String parameters can be swapped without any compiler warning (for example email and phone). The Builder makes every call self-documenting, lets us skip unwanted optional fields, and centralizes validation, so an invalid account (no id, e-mail without @, unknown level) can never exist.

If we did not use it. We would need either a telescoping constructor (many overloads) or a no-argument constructor with setters. The second option allows half-built, unvalidated objects.

```
new StudentAccount.Builder("20240002", "Sara Khelifi", "sara@univ.dz")
    .phone("0555123456").level("M1").major("Software Engineering")
    .scholarship(true).addCourse("Design Patterns").build();
```

3.2 Singleton - AccountRepository and AuditLogger

Where? Two classes: `AccountRepository` (stores every account in a map and enforces the unique id) and `AuditLogger` (records every operation). Both have a private constructor and a static `getInstance()`, implemented with the *initialization-on-demand holder* idiom: the instance is created lazily, on the first call, and the JVM guarantees thread safety without synchronizing getInstance(). Their methods are `synchronized` because they are shared mutable state.

Problem solved. The program must have one repository and one audit log. If every part of the program created its own, accounts would be scattered among several copies, uniqueness of the id could not be checked, and the audit trail would be split. The demo proves it: *r1 == r2* is true, and accounts saved through one reference are seen through the other.

If we did not use it. Passing the same object everywhere by hand is fragile, because one forgotten *new* silently creates a second repository, and the bug appears only as "missing" accounts.

3.3 Prototype - Prototype<T> and StudentAccount.copy()

Where? The interface `Prototype<T>` declares `copy()`; `StudentAccount` implements it. A template account is built once for a batch (for example L2, Computer Science, courses Algorithms / Databases / Java OOP). For each newcomer, `copyFor(id, name, email)` clones the template and changes only the identity (the phone number is reset, because it is personal).

Problem solved. Hundreds of students join the same level and major with the same courses every year. Rebuilding each account from scratch repeats the same level, major and course list hundreds of times. Cloning avoids this repetition. The copy is **deep**: the course list is duplicated, so adding "Machine Learning" to one student changes neither the template nor the other students (shown in Figure 3).

If we did not use it. We would copy the shared data by hand each time, with a risk of forgetting a field. A shallow copy would be worse: all students would share the same course list object, and one modification would leak to everybody.

4. Execution results

The screenshots below show the real output of `java com.uniaccounts.Main`.

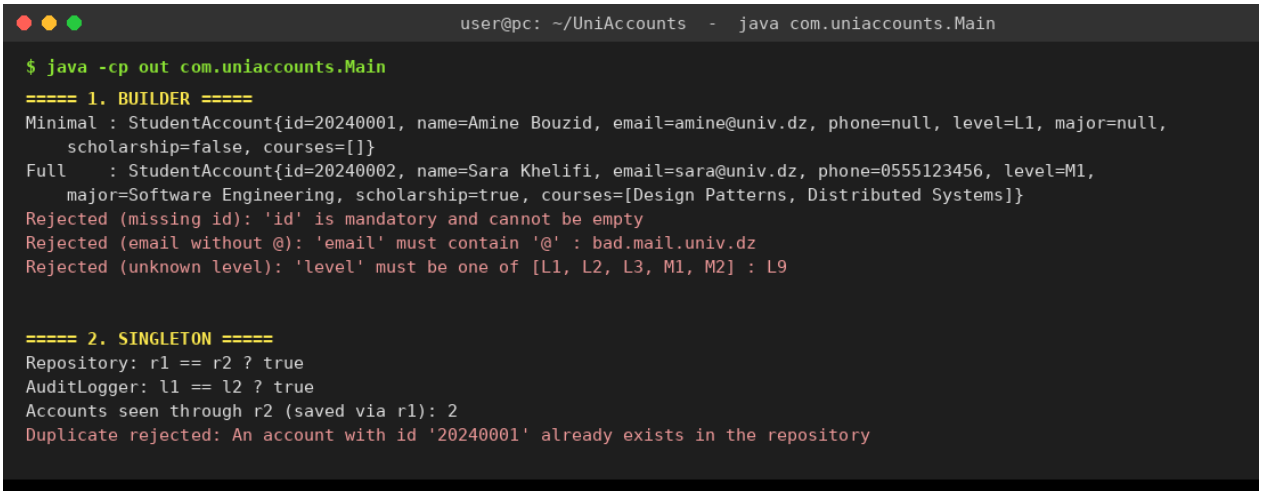


Figure 2 - Builder (defaults, optional fields, rejected invalid data) and Singleton (same instance, duplicate id rejected).

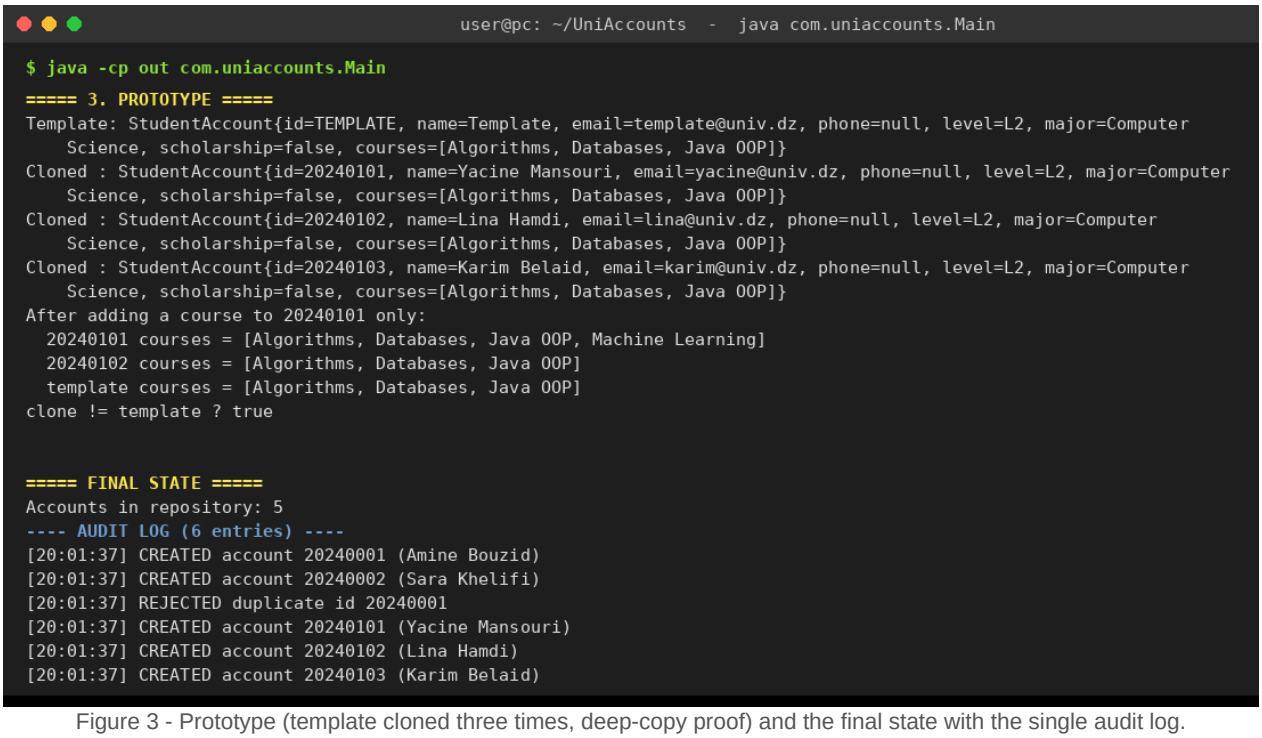


Figure 3 - Prototype (template cloned three times, deep-copy proof) and the final state with the single audit log.

5. Conclusion

The three creational patterns work together: the **Builder** guarantees valid accounts, the **Prototype** produces batches of them cheaply, and the **Singleton** classes give them one place to live and one place to be audited. This core is the base that Work 2 will extend (state, observer, composite, adapter, etc.) without rewriting it.

Sources: course lecture notes on design patterns; Gamma, Helm, Johnson, Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*; J. Bloch, *Effective Java* (Builder, Singleton items).